14. Consumer Setup - Part3 (Other configurations)

Wednesday, 18 March 2026 2:56 PM

Step1:
Consumer Starts and want to join Group

Doubt: can a consumer join more than 1 group?
My Answer: No, 1 consumer can join only 1 group.

Step2:
Finds the Partition number of topic "_consumer_offsets"

hash(group.id) % N

Step3:
Consumer requests metadata (first time or refresh)

In this consumer invoke one of the bootstrap server(broker) and ask for metadata. And cache it.

Doubt: how frequently this metadata is refreshed?
My Answer: Multiple factors which makes consumer initiates the metadata refresh.

- Periodic refresh
  - application.properties
  - #default 5 minutes
  - spring.kafka.consumer.properties.metadata.max.age.ms=300000
- On demand refresh
  - Invoked broker, but broker returns NOT_LEADER_FOR_PARTITION
  - Information missing in Cache
  - etc.

Step4:
Finds Group Coordinator and Send JoinGroupRequest

Request contains:
- Consumer Group it need to join
- Topics it want to subscribe

Doubt1: can 1 consumer request to join more than 1 topic?
Doubt2: can multiple consumers in a group request to join different topic?
Doubt3: can multiple consumers in a group request to join same topic?

All these doubts I have covered in Part-2 of Consumer Setup *(kindly check it out, if there is any doubt with above points)*

Step5: Partition assignment

Broker3 (Group coordinator)
Group: **order-consumer-group**

Topic: order-events

Consumer1 : handle Parition-0
Consumer2: handle Partiton-1
Consumer3: handle Partition-2

Doubt: How partitions are divided among consumers in the group

Range Assignor (Default)

Consumer: C1, C2, C3
Topic: order-events
Partitions: P0, P1, P2, P3, P4, P5

- Lays out Partition in numeric order.
- Lays out Consumer in Alphabetical order.

perform:
Logic: partitions/consumers = 6/3 = 2

C1: P0,P1
C2: P2, P3
C3: P4, P5

Round Robin

Consumer: C1, C2, C3
Topic: order-events
Partitions: P0, P1, P2, P3, P4, P5

- Just distribute the partitions 1 by 1 to
- all consumers.

C1: P0,P3
C2: P1, P4
C3: P2, P5

Sticky Assignor

Consumer: C1, C2, C3
Topic: order-events
Partitions: P0, P1, P2, P3, P4, P5

- Starts with Round Robin assignment.

C1: P0,P3
C2: P1, P4
C3: P2, P5

- But its main advantage comes during Rebalance.
When any new Consumer is added or existing consumer removed from the group.

When rebalance happens:

C1: P0,P3
C2: P1, P4
C3: P2, P5

Say C3 got crashed

Range Assignor (Default):
Round Robin:

- Every Consumer (C1, C2) drops the partition they were allotted.
- Every Consumer stops reading.
- Then algorithm runs full recalculation from start for remaining 2 Consumers.

Ex:
C1: P0,P2, P4
C2: P1, P3, P5

Problem with this approach:

earlier C1 has P0 and P3 and after rebalancing, P3 is removed and P2 and P4 is added.

So C1 has to do all of cleanups like cache clearance of P3 and maintains new offset of P2, P4.

But with

Sticky Assignor:

- It is more smarter, during rebalance, it try to achieve the even distribution(as much as possible).
- Also during rebalance, it tries to keep the consumer attached to the partition they already had.

C1: P0,P3, P2
C2: P1, P4, P5
C3: P2, P5

Cooperative Sticky

Range Assignor (Default):
Round Robin:
Sticky Assignor:

All are Eager Rebalancer,
means when rebalance happens, Consumers stop reading from the partitions.

Till new enrollments are assigned.

But with Cooperative protocol,

Step1: Algorithm decides, old partitions which Consumers will still hold.

Step2: Consumers continue reading from those partitions, and only partitions which are getting removed from them or newly added to them is something which consumer later update.

Say C3 got crashed, so it picks the partitions allotted to C3 and tries to distribute evenly to the available consumers.

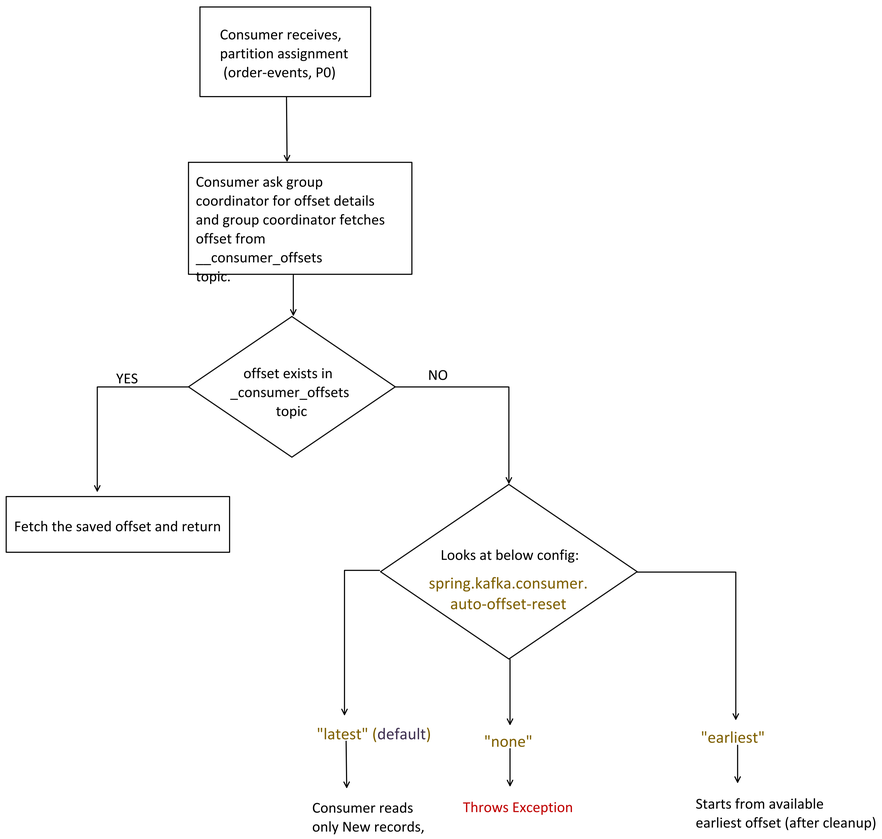
So here if you see, C1 and C2 still hold the partitions which were previously allotted.

spring.kafka.consumer.properties.partition.assignment.strategy=org.apache.kafka.clients.consumer.RangeAssignor
or
spring.kafka.consumer.properties.partition.assignment.strategy=org.apache.kafka.clients.consumer.RoundRobinAssignor
or
spring.kafka.consumer.properties.partition.assignment.strategy=org.apache.kafka.clients.consumer.StickyAssignor
or
spring.kafka.consumer.properties.partition.assignment.strategy=org.apache.kafka.clients.consumer.CooperativeStickyAssignor

Step6 and 7:
Consumer assigned P0 of topic order-events

Consumer ask Group Coordinator till where the offset is read for P0 of topic order-events

Doubt: What if there is no offset info present?



after the partition
assignment

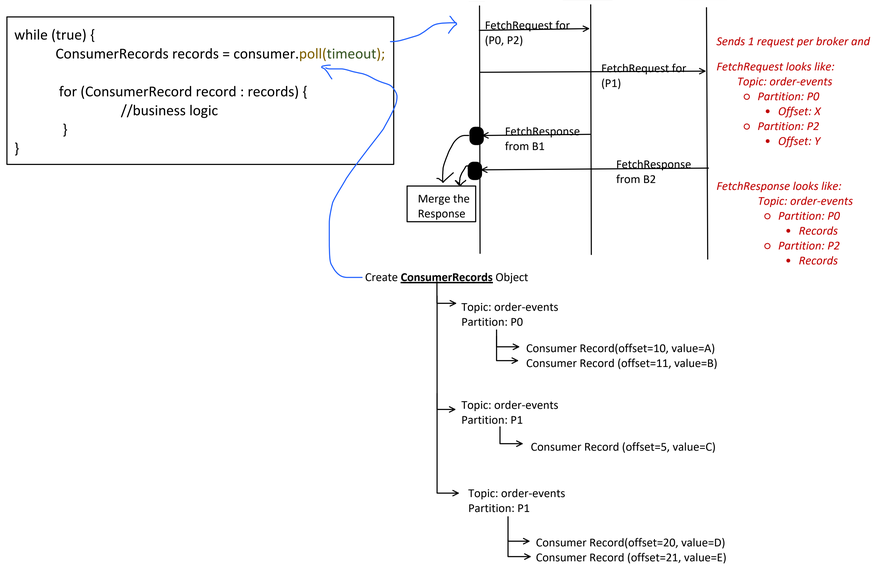
Step 8 and 9:
Check the metadata and invokes Leader Broker of
Topic "order-events"
Partition-0
offset: xyz

To fetch from given offset fetched in previous step

Doubt: What if there is no event? How much time consumer will wait?

- Kafka Consumer is single threaded, it process the records one by one.
- During poll(timeout), we can also set the timeout, so consumer will wait max this much time for the brokers to response else it will proceed with whatever response it received or empty list.

```
#5seconds
spring.kafka.listener.poll-timeout=5000
```



Doubt: in one Poll() how much data is sent?

- We know that consumer can ask a Broker to return from multiple partitions and topics in 1 single request.

But how much data broker will send in 1 Fetch Request?

FetchRequest:
- *Topic: A*
  - *Partition: P0*
    - *Offset: X*
  - *Partition: P1*
    - *Offset: Y*
- *Topic: B*
  - *Partition: P1*
    - *Offset: Z*

Broker

Topic-A: Partition-P0, Partition-P1

Topic-B: Partition-P1

Fetch records From offset-X (<=1MB)

Fetch records From offset-Y (<=1MB)

Fetch records From offset-Z (<=1MB)

Max data the Broker returns PER partition (1MB)
spring.kafka.consumer.properties.max.partition.fetch.bytes=1048576

max.partition.fetch.bytes = from each partition, this is the max number of bytes we can fetch

These total bytes should be <= 52MB

Maximum data (total) the Broker can return in one request (52MB)
spring.kafka.consumer.properties.fetch.max.bytes=52428800

fetch.max.bytes = combined total MB fetched from each partition

Minimum data (in bytes) the Broker must have before responding
spring.kafka.consumer.properties.fetch.min.bytes=1

How long the Broker can hold the request to meet fetch.min.bytes (0.5sec)
spring.kafka.consumer.properties.fetch.max.wait.ms=500

Broker decides when to respond:

Condition1:
Data accumulated >= min.bytes
Send response to consumer

Condition2:
Time out reached
fetch.max.wait.ms (500ms) -> send anyway whatever data is fetched

Max number of records returned in a single poll() call
spring.kafka.consumer.properties.max.poll.records=500

Send response to Consumer

Note: response from broker is totally limited by BYTES not by number or records.

So its possible that:

max.bytes = 52MB is sent by broker but it might contains 2000 Records.

But then what about below config?

spring.kafka.consumer.properties.max.poll.records=500

Consumer receives data
Internal buffer contains > 500 records

Now Consumer applies:
spring.kafka.consumer.properties.max.poll.records=500

And return <=500 records to poll().

Remaining records will stay in buffer

↓

When Next Poll request comes, it will first check buffer, if records present there it will send that and not make network call to Broker

Doubt: Is it possible to have gap between polls()

```
# Sleep time between two poll() calls (0ms is default, means as soon as possible)
spring.kafka.listener.idle-between-polls=0
```

Doubt: how to know if Consumer is stuck while processing the events or any other issue with Consumer?

```
# with this consumer sends Heartbeat Pulse to Group coordinator every 3 Seconds
spring.kafka.consumer.properties.heartbeat.interval.ms=3000


# Max time Group coordinator will wait for the heartbeat (generally its higher than heartbeat.interval.ms)
#after this say 45s it will consider Consumer as dead and initiate rebalance
spring.kafka.consumer.properties.session.timeout.ms=45000
```

```
# Group Coordinator expects 1 poll() call every 5 mins,
if say consumer took 6 minutes to process records
# group coordinator will think that Consumer got stuck
#initiates rebalance
spring.kafka.consumer.properties.max.poll.interval.ms=300000
```

Step 10:

Consumer process events

Doubt: What if exception comes during Deserialization?
Doubt: What if exception comes during processing the record?

Already covered in previous video, kindly have a look if there is any doubt with this.

Step 11:

Offset commit

Doubt: How to manage auto commit?

For auto-commit

```
#Kafka commits offsets every 5 seconds (background thread)
spring.kafka.consumer.enable-auto-commit=true
spring.kafka.consumer.properties.auto.commit.interval.ms=5000
```

For manual-commit

```
spring.kafka.consumer.enable-auto-commit=false
spring.kafka.listener.ack-mode=
```

batch (default)

- poll() get 500 records
- Process all 500
- Commit once

Cons:
- If say last record from batch failed Entire batch retried.

record

- Process record1 -> commit
- Process record2 -> commit

Cons:
- More commit overhead

time

```
spring.kafka.listener.ack-mode=time
spring.kafka.listener.ack-time=5000
```

Commit every 5 sec

count

```
spring.kafka.listener.ack-mode=count
spring.kafka.listener.ack-count=10
```

Commit after every 10 records

manual

```
@KafkaListener
public void process (ConsumerRecord rec,
                     Acknowledgment ack) {
    process(rec); //business logic
    ack.acknowledge(); // commit
}
```

Even though after each record, I am committing but internally it will just wait for poll() to finish and send one commit request to kafka.

manual_immdediate

```
@KafkaListener
public void process (ConsumerRecord rec,
                     Acknowledgment ack) {
    process(rec); //business logic
    ack.acknowledge(); // commit
}
```

For each call, it will send commit request.